LMCS — What to say on demo day

Team HexTech · Problem Statement SIH26034 · Legal Metrology Compliance Scanner · Reading time about 7–8 minutes

Plain text in this colour is meant to be spoken. Grey notes are for you, not the judges.

1. The problem (0:00–0:50)

Good morning. We are team HexTech, and our problem statement is SIH26034, from the Department of Consumer Affairs.

Every packed product sold in India must print certain details on its label. The law is the Legal Metrology (Packaged Commodities) Rules, 2011. These details are: who made or packed it, the net quantity, the MRP, the date of packing, the best-before date, and a customer care contact. They are called mandatory declarations, and they protect the buyer.

The problem is checking them. India has lakhs of packed products, in shops and online. An inspector has to pick up each pack, work out which rules apply, read the small print, see what is missing, take photos and write a report. It is slow, and two inspectors can reach different conclusions.

There is a second problem. A label can be complete and still be wrong. If a shop sticks a new, higher price over the printed MRP, every required field is still on the pack. Software that only asks "is the field there?" will pass it.

So our problem statement asks for a system that scans labels and product images, finds the mandatory declarations, checks whether they are correct, complete and readable, and gives reports and dashboards to enforcement officials.

2. What we built (0:50–1:40)

We built LMCS, the Legal Metrology Compliance Scanner. It is working software, not a design. It has three parts: a mobile app for consumers and inspectors, a server that runs the compliance rules and stores the evidence, and a web dashboard for the Controller of Legal Metrology.

The flow in one line: a consumer scans a pack, the app reads the label, the server runs 13 checks and compares the pack with a product registry. If something is wrong, a complaint goes to the inspector with the photo as evidence. The inspector verifies it and files a report, and the Controller confirms it and issues a show-cause notice as a PDF.

- **Consumers** scan a pack in a shop, get an answer, and a complaint is filed for them if something is wrong.
- **Inspectors** get those complaints on a map, can re-scan the pack on site, and file a report.
- **The Controller (admin)** reviews the evidence on the dashboard and is the only one who can issue a notice.

3. The 13 checks (1:40–3:10)

Every scan runs all 13 checks, and each one names the rule it comes from.

#	Check	Rule
1	Manufacturer, packer or importer name and address	Rule 6(1)(a)
2	Country of origin, for imported goods	Rule 6(1)(aa)

3	Common or generic name of the product	Rule 6(1)(b)
4	Net quantity	Rule 6(1)(c)
5	MRP inclusive of all taxes, and the unit sale price	Rule 6(1)(e)
6	Month and year of manufacture or packing	Rule 6(1)(d)
7	Best before / use by date	Rule 6(1)(da)
8	Consumer care details	Rule 6(2)
9	Minimum letter height (font size)	Rule 7(2) + Table I
10	Legibility: print contrast	Rule 9(1)
11	Placement on the display panel	Rule 8(1)
12	LMPC registration number	Rule 27
13	GSTIN format and checksum	CGST Rule 46

Twelve of these come from the Packaged Commodities Rules. The thirteenth, GSTIN, comes from GST law, because a GSTIN is required on invoices, not on labels. So we check it only when a pack prints one, and we never fail a pack for leaving it out.

Each check gives one of four results:

- **PASS** — the requirement is met.
- **FAIL** — it is clearly broken.
- **REVIEW** — we could not establish it, so a person must look.
- **EXEMPT** — the rule does not apply, like country of origin for a product made in India.

Only a FAIL can go into a violation report, plus one more ground: a registry mismatch in MRP, quantity or maker, cited under Rule 6(3), and only after the inspector ticks that they checked the pack themselves, because a misread photo looks exactly like tampering. REVIEW never becomes a notice. That is why our system does not accuse honest companies.

4. What is new in our work (3:10–4:20)

First: the registry cross-check, which catches tampering.

The barcode is not just detected, it is used. We look up the barcode in a product registry and compare the printed MRP, net quantity, FSSAI licence, GSTIN, veg mark and manufacturer with the registered values. If the label contradicts the registry, the scan is flagged as suspected tampering, even though the label looks complete. A price sticker raising the MRP is caught here. A lower MRP is not flagged, because Rule 6(3) allows a sticker that lowers it. No label-checking tool we compared does this.

We do not compare dates or batch numbers, because those change with every batch. Dates are checked for format and expiry instead. A licence number read one digit off is treated as an OCR misread, not tampering.

Second: the barcode as a ruler.

A photo has no sense of real size. An EAN-13 barcode is printed to a world standard, 37.29 mm wide. So we measure the barcode in pixels, work out millimetres per pixel, and convert the height of the printed text into millimetres. That is how the minimum font size rule is measured from an ordinary phone photo.

Third: honest results.

If a heading is read but its value isn't, or the scale looks impossible, we return REVIEW, never a guessed PASS or FAIL. Font size never FAILs from a photo: a reading under the limit is REVIEW, and a person measures it on the pack. Our placement check is labelled openly as an approximation: it checks whether declarations are grouped together, not true front-panel detection.

Fourth: evidence that holds up.

Every photo gets a SHA-256 fingerprint when it is sent, with the GPS location and the time checked against an internet time server, so we do not have to trust the phone's clock. A scan cannot be edited after it is sent. A re-used photo is flagged, and any field the user corrected is recorded and shown.

Fifth: two people, not one.

The inspector files the report, and only the Controller can confirm it and issue the notice. The server refuses an inspector who tries to issue a notice, and refuses a notice before confirmation. A database lock makes sure a double click cannot create two notices.

5. Technology, and where each part is used (4:20–6:00)

On the phone — Flutter

- **Flutter** — one Android app for both consumers and inspectors, so there is one codebase.
- **Google ML Kit Text Recognition v2** — reads all the text on the phone itself, and gives the position of every line.
- **Google ML Kit Barcode Scanning** — reads the barcode number and where it sits in the photo.
- **Our field extractor (our code)** — turns loose text into fields. OCR returns headings in one column and values in another, so we match each heading with the value beside it by position. It also handles real OCR mistakes we hit: "Rs.109,00" with a comma, "BATCH N0." with a zero, "NET QUANTIT" where the E came out accented and the end was cut off, and dates like "11/JUN25" with no separator.
- **geolocator** — the GPS location. **ntp** — the true time from pool.ntp.org.
- **Dio** — sends the photo and readings to the server. **Secure storage** — keeps the login token safe.

On the server — Python and FastAPI

- **FastAPI** — receives every scan and checks the user's role on every request.
- **OpenCV and NumPy** — Otsu's method separates ink from paper to measure print contrast, and shape-and-colour analysis finds the FSSAI vegetarian mark. That detector refuses to guess: an older colour-only version once reported a bedsheet pattern as a non-veg mark, so now the mark must have its dot, an unbroken square outline and a light background before we call it. Font size and placement don't use OpenCV: they are worked out from the sizes and positions of the text boxes.
- **pyzbar (ZBar) — the backup barcode reader.** ML Kit reads the barcode on the phone, but a barcode can be missed if the photo is soft or the label is glossy. Without a barcode there is no registry check and no ruler for font size. So when the phone reports no barcode, the server tries again: pyzbar reads the uploaded photo once, exactly as it was sent. We record who read it, "device" for the phone or "server" for pyzbar, and show that on the result screen.
- **Our compliance rules (compliance_rules.py, our code)** — runs the 13 checks and returns PASS, FAIL, REVIEW or EXEMPT with the rule citation.
- **Our registry cross-check (our code)** — the barcode comparison described earlier.
- **SHA-256, JWT and bcrypt** — photo fingerprints, login tokens, hashed passwords. Public sign-up creates consumer accounts only; inspector and admin accounts are created by an administrator.

Data, maps and documents

- **PostgreSQL** — stores everything; row locks stop duplicate notices.
- **SQLAlchemy and Alembic** — the database models and 8 tracked schema changes.
- **React with Vite** — the Controller's dashboard: report queue, evidence review, search, and PDF and Excel downloads.
- **Leaflet, flutter_map and OpenStreetMap** — complaints and reports on a map, on the dashboard and on the inspector's phone.
- **ReportLab** — two different PDFs: the compliance report for a scan, with the photo, and the GO show-cause notice for a confirmed violation.
- **pytest and flutter test** — 333 backend tests and 97 mobile tests that run automatically.

6. What the database stores (6:00–6:40)

Everything is in PostgreSQL. The main tables are:

Table	What it holds
users	Name, email, role (consumer, inspector or admin), hashed password.
evidence_records	One row per scan: the photo file, its SHA-256 fingerprint, size and type; GPS latitude, longitude and accuracy; the phone's time, the internet time and the difference; the raw OCR text; the fields we extracted; who scanned it; the shop name and address; and the audit links — which scan this is a rescan of, whether the same photo was sent before, and which fields the user changed.
rule_checks	One row per check per scan: rule number, name, the law cited, the result, what was detected, what was required, and why it failed.
compliance_results	The overall verdict for a scan: compliant, needs review or action needed, the reason, the registry result, the veg-mark result, and how many checks failed.
complaints	Complaint number, the consumer, the product, the shop, the status, the location.
violation_reports	Report number, the scan, the inspector, the admin, the status, the severity, the rules cited, and the inspector's and admin's notes.
legal_notices	Notice number, who it is addressed to, the issuing authority and officer, the clauses cited, the notice text, the days to reply, and the PDF's own fingerprint. One notice per report is enforced by the database itself.
product_registry	The trusted product data: barcode, name, manufacturer and address, MRP, net quantity, FSSAI licences, GSTIN, and whether it is vegetarian.
rules	The 13 rules with their legal references and severity, listed on the dashboard. The Controller can switch a rule off and new scans follow it; editing and adding rules is our next step.

Nothing is deleted. A case is closed with a note, so the history stays.

7. The app and the dashboard (6:40–7:20)

In the app, a consumer logs in, presses scan, and takes a photo. The next screen is Review Details: the app shows what it read, and the user can correct any field. Every change is recorded. If the MRP or the dates are not on that face, the app says where the label says they are, for example "see seal", and offers "Scan another part" to photograph the lid, the seal or the veg mark. It fills the blank fields, asks before replacing a value it reads differently, and the photo goes with the scan so the server looks for the veg mark on it too. The result screen then shows COMPLIANT, NEEDS REVIEW (a required detail, such as the MRP or a date, couldn't be read, so a person checks the pack) or ACTION NEEDED, the 13 checks, the Authenticity card with the registry result, the Barcode card showing who read it, and the complaint number if one was filed.

Consumers see only their own scans. Inspectors have the complaint list, the map, the full scan history, a New scan button, and a report form.

On the dashboard, the Controller sees the queue of reports, the map, and each report in detail: the evidence photo and any extra photos of other sides, a live fingerprint check that proves each photo has not changed since capture, the GPS and time, and the rules cited. From there they confirm or reject, and issue the GO notice PDF. There is also a System Log with search by product, barcode, scan number, complaint number or user, with Excel and PDF export, and a Rules page listing the 13 checks.

8. Where we stand, and closing (7:20–8:00)

We are at TRL 4: a working prototype, tested by our team. We have 333 backend tests and 97 mobile tests, and we tested on 31 scans of real packs, 5 products in all.

We measured ourselves honestly. In the first batch, only 3 of 7 genuine packs were read correctly. Each false alarm was traced to its cause, fixed, and given a test. By the third batch it was 6 of 8. We also planted 2 violations, and the system caught both. It is a small sample, so we do not call it a benchmark.

Our registry has 8 products, 3 of them entered from real packs. In real use it would be connected to GS1 India's DataKart, where brand owners already keep their product details; the Smart Consumer app of the Department of Consumer Affairs already reads it by barcode.

Next steps: connect the registry to DataKart, add Hindi and regional labels, an offline mode for inspectors, cloud hosting, then a pilot with a Legal Metrology office, a trained FSSAI-mark detector, and checks on e-commerce listings.

Closing line: "Checking whether the declarations are present is not enough, because a tampered label still looks complete. LMCS checks whether they are true, keeps evidence that can stand up later, and when it is not sure, it says so and lets a person decide."

9. Answers for the SIH evaluation criteria

Judges score ideas on these parameters. Each answer below is short enough to say out loud.

Novelty — how is it innovative?

> *"Most label tools stop at reading the label. We do three things they do not. One, we use the barcode to look the product up in a registry and compare the printed MRP, quantity, licence, veg mark and maker, so a tampered label that looks complete is still caught. Two, we use the barcode as a physical ruler, because it is printed to a standard 37.29 mm, which lets us measure letter height in millimetres from an ordinary photo. Three, when we cannot establish something we return REVIEW instead of FAIL, so an honest company is never accused by a blurry photo."*

Complexity — how hard was it?

> *"Three parts working together: an Android app, a Python server and a web dashboard. Thirteen compliance rules, each citing its rule. Our own computer vision for the veg mark, print contrast and font size. Our own field extractor that matches headings to values by position and survives real OCR mistakes. A database lock so two clicks cannot create two notices. 333 backend tests and 97 mobile tests."*

If asked why no trained model: "We used proven on-device OCR and spent our effort on the hard part, which is applying the law correctly and not accusing honest packs. A trained FSSAI-mark detector is on our roadmap."

Feasibility — can it actually be built and run?

> *"It is already built and running end to end, and you are about to see it on a real pack. It needs no special hardware: an ordinary Android phone, a normal server and a browser. The reading happens on the phone, so the server does little work per scan. The whole stack is open source or free: Flutter, Python, FastAPI, PostgreSQL, OpenCV and Google ML Kit on-device. We are at TRL 4, tested by our team on 31 scans of real packs (5 products), with 430 automated tests."*

- **Cost:** no cloud OCR bill, because OCR runs on the phone. Hosting is one small server and a database.
- **Time:** the prototype was built and tested in four weeks, in the weekly plan in our proposal.
- **Deployment:** containers on government or commercial cloud, managed PostgreSQL with daily backups, the dashboard over HTTPS, and the app given to officials through the Play Store. This is section 4.4 of our proposal.

Compatibility — what does it work with?

- **Phones:** any modern Android phone with a camera. Nothing else to install on the pack side.
- **Dashboard:** any modern browser, no installation.
- **Server:** plain Python, runs on Windows or Linux, and in a container on any cloud, so there is no lock-in to a single provider.
- **Packs:** any pack carrying a standard retail barcode (EAN-13, EAN-8 or UPC). Packs without a barcode still get all the text checks; only the registry check and the ruler need the barcode.
- **Data:** the registry's main fields (MRP, net quantity, maker) are ones GS1 India's DataKart already holds, so connecting to it needs no redesign.
- **Network:** reading happens on the phone; only sending needs a connection. An offline queue is Phase 2.
- **Languages:** English labels today; Hindi and regional labels are Phase 2.

Practicability — will it work in the field?

"It is built around what really happens in a shop. Photos are blurry, glossy or taken at an angle, so anything we cannot read goes to REVIEW, not FAIL. Dates are often on the lid and the veg mark on another side, so the app says where to look and lets you add photos of other parts, which are checked too. The user can correct a misread field before sending, and every correction is recorded. On our latest batch, 6 of 8 genuine packs were read correctly, and every miss we found has been fixed and given a test."

Sustainability — who keeps it running?

"The Department of Consumer Affairs or the state Legal Metrology departments would run it on government cloud. Each check is stored with the law it comes from, the Controller can switch a rule off from the dashboard, and editing and adding rules there is our next step. Product data comes from GS1 India's DataKart, which brand owners keep up to date themselves. And because the phone does the reading, there is no per-scan cloud cost."

Scale of impact — who benefits?

- **Consumers** get a way to report a wrong label with real evidence, instead of only a complaint form.
- **Inspectors** receive only cases with evidence, already on a map, with the report filled in.
- **The Controller** decides from the photo, the fingerprint check, the GPS and the time, on one screen.
- **Honest manufacturers** are protected, because uncertainty becomes REVIEW and two people must agree before any notice.
- It can grow one district at a time, and the same checks apply to every packed product in India.

User experience

"A consumer does one thing: take a photo. Everything else is filled in for them, and they only correct what is wrong. An inspector never has to retype the case; the report arrives with the failed checks already cited. The Controller sees one screen with the photo, the fingerprint check, the location and the time."

Future potential

"Phase 2 is connecting the registry to GS1 India's DataKart, Hindi and regional labels, an offline mode, and cloud hosting. Phase 3 is a pilot with a Legal Metrology office, a trained FSSAI-mark detector, and e-commerce listings, where we would take the product link, apply the online rules, and save a page snapshot as evidence."

10. Questions judges may ask

How does pyzbar fit in? Isn't ML Kit enough?

ML Kit reads the barcode on the phone and usually succeeds. When it does not, because the photo is soft or the pack is glossy, the server reads the uploaded photo again with pyzbar. The barcode matters because it unlocks the registry check and the font-size ruler. We record which reader found it, and show that on screen.

How can you measure font size from a photo?

An EAN-13 barcode is a standard 37.29 mm wide. We measure it in pixels, get millimetres per pixel, and convert the text height. If the scale looks impossible, or the reading is borderline, the result is REVIEW.

Why is GSTIN not checked on this pack?

GSTIN is not a required label declaration; it is required on invoices under GST law. We check it only when a pack prints one: the 15-character format and the checksum. We never look it up in the live GST database, and we say so.

You did not catch the fake batch number or date.

Dates and batch numbers change with every batch, so they are not in the registry. Dates are checked for format and expiry instead.

Is the placement check real?

It is an approximation, and the result says so. It checks whether declarations are grouped together. True front-panel detection is our next step, and it would also give the font-size rule its correct limit, because the limit depends on the panel area.

What about e-commerce listings?

Our prototype is built for physical packs, which is where inspectors work. You can upload a screenshot and it reads the text, but font size and contrast do not apply to a web page, and the online rules list different declarations. Full listing checks are Phase 3.

Did you train any AI model?

No. We used proven on-device OCR and spent our effort on applying the law correctly. A trained FSSAI-mark detector is on our roadmap.

What if there is no internet?

The reading happens on the phone. Sending the scan needs a connection. An offline mode that queues scans is in Phase 2.

Can evidence be faked?

Every photo is fingerprinted when sent, the time is checked against internet time, a sent scan cannot be edited, re-used photos are flagged, and the dashboard re-checks the fingerprint when the Controller opens the case.

Is 75% good enough?

It is a small sample of 31 scans, and we report it as it is. What matters is the direction: every false alarm was traced, fixed and given a test, and the system never turns an uncertain reading into a violation.

Where does the registry data come from?

Today we have 8 products, 3 entered from real packs. For real use, GS1 India's DataKart, the national product data repository, already holds brand owners' product details, and the Smart Consumer app of the Department of Consumer Affairs already reads it by barcode. LMCS would connect to the same source. If a product is not there, all 13 checks still run; it just shows as not verified.

What if the barcode is printed bigger or smaller?

Barcodes may be printed from 80% to 200% of the standard size, and our ruler assumes 100%, so we call the size check a first version. A smaller barcode makes letters read large, so we may miss small print; a bigger one could make them read small, which is why a low reading is only ever REVIEW, never FAIL. A printed scale card next to the pack is a possible fix.

How often does REVIEW happen?

On our 31 real pack scans, a third of all checks were REVIEW. A REVIEW never creates a complaint, so it never reaches the inspector. Almost half come from two checks one photo cannot settle: the registration number and true print contrast.

Timing tip: sections 1–4 are the story, about four minutes. Sections 5–7 are the depth, about three minutes. Section 8 is the close. If you are short of time, shorten section 5 to the three headings — phone, server, data — and keep the pyzbar explanation only if someone asks.